Week 12

ID1063 Introduction to Programming in C

Lectures 21–22: Dynamic Memory Allocation, Structs, Problems

malloc : Dynamic memory allocation

- **Purpose:** malloc requests a block of memory from the *heap* memory

- **Must include** `#include<stdlib.h>`

- **Syntax:**

```
int count;
int *ptr = malloc(count* sizeof(int))
```

Recap: Returning pointers from Functions

- **Syntax:** The return type is a pointer type

```
int* find_max(float x, int n)
```

- Why do this: The function can hand back *where* something is, not just *what* is it

- **The pointer** must point back memory that still exists after the function returns

Memory management caution

- Pointers should not return addresses of variables *local* to the function, since these are destroyed when it returns
- A pointer to a destroyed variable is a **dangling pointer**.
- Using it is undefined behaviour: you may get garbage, a crash, or output that seems to work and then fails later.

```
int* make_squares(void) {
    int sq[5];
    for (int i = 0; i < 5; i++)
        sq[i] = i * i;
    return sq;          /* BUG: sq disappears when the
function returns */
}
```

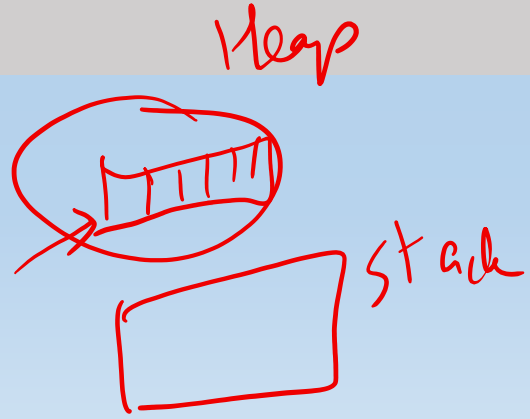
How to fix?

- Option 1: Make the array in the caller function
  - Use the function only to fill in the array
- Option 2: Use heap memory
  - Memory from malloc lives on the **heap**. It is not destroyed when the function returns; it stays until someone calls free.
  - So a function can return a pointer to it safely.
  - The **caller becomes responsible for calling free** on it.

Heap memory in functions

```
int* make_squares(int n) {
    int *sq = malloc(n * sizeof(int));
    if (sq == NULL)
        return NULL;           /* if allocation failed */
    for (int i = 0; i < n; i++)
        sq[i] = i * i;
    return sq;                 /* safe: heap memory outlives the fn */
}
```

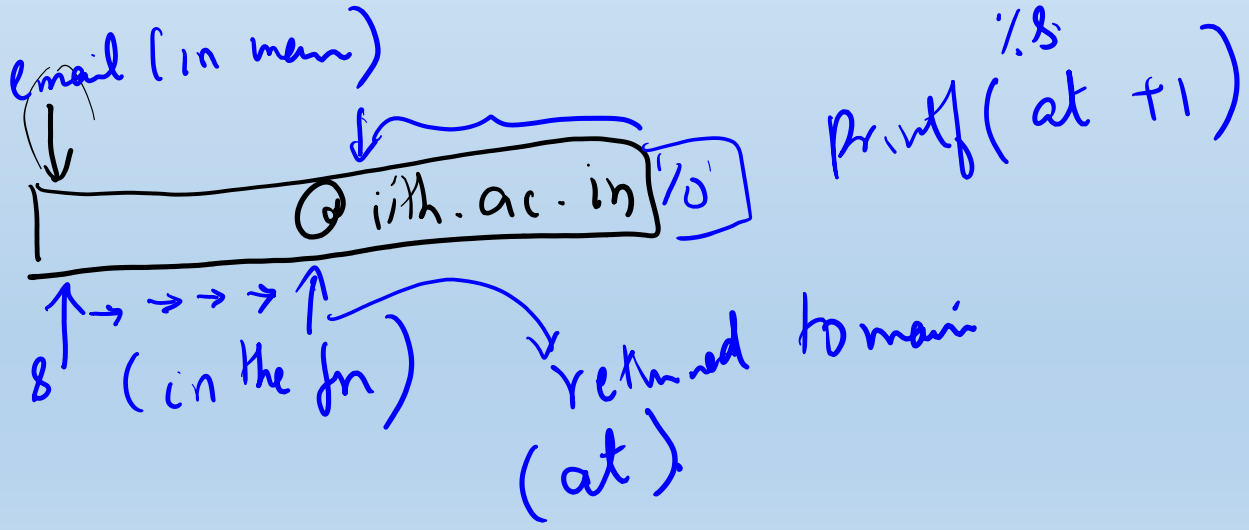
on heap memory.



Example 3: Strings, detecting email

- Write a function to find if a given string is an email address
  - Check for the presence of '@'

- The function will return NULL, to indicate that it is not an email address



`calloc`: allocate and clear

- Signature: void *calloc(count, size);

```
int *a = malloc(5 * sizeof(int));   /* contents: garbage */
int *b = calloc(5, sizeof(int));    /* contents: 0 0 0 0 0 */
```

- Unlike malloc, it **sets every byte to zero**, so an int array starts with all elements equal to 0.
- It returns NULL on failure. Needs to be freed later.

- Use it when you need a fresh array of counters, flags, or totals that must start at zero.

Example use of calloc

- Useful for frequency counters, running totals, etc.
  - Anything that requires a zero initialization

E.g.: Count the frequency of digits in an array

```
int *counts = calloc(10, sizeof *counts);

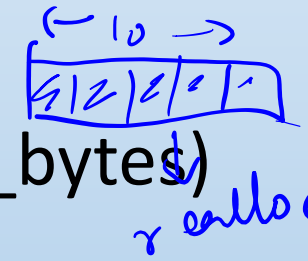
if (counts == NULL)
    return 1;

int digits[] = {2, 0, 2, 6};
for (int i = 0; i < 4; i++)
    counts[digits[i]]++;
printf("Number of 2s: %d\n", counts[2]);  /* 2 */

free(counts);
```

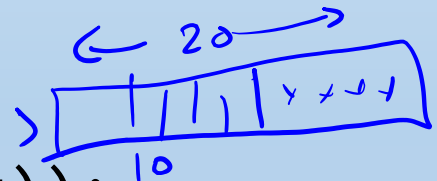
`realloc`: Resize an existing block

- Changes the size of storage obtained using malloc, calloc, or realloc.
  - Useful when an array needs more space or unused space can be released.



- **Syntax:** realloc(pointer, new_total_size_in_bytes)
- Example:

Suppose values is a pointer/array of ints



```
int *newtemp = realloc(values, 6 * sizeof(int));
```

realloc: Properties



- Existing data is preserved up to the smaller of the old and new sizes.
- The allocation may move to a different address.
- Additional space after growing is **uninitialized**.
- Should be freed just like malloc, after use
- **Uses:** Expanding a list of readings; extending a string buffer; shrinking a completed record list.

realloc: Cautions

- **Use a temporary pointer:** allocation can fail.
  - On failure, the original allocation remains valid.
  - On success, use the returned pointer; earlier pointers into the allocation may be invalid.

```
/* Unsafe: failure loses the original address */
values = realloc(values, 6 * sizeof(int));
```

- Supply the **new total size**, not just the extra size.
- When shrinking, discarded elements are lost.
- Use a positive size; use free to release storage completely.

Structs

(compound types)

Student Records

- A student has a roll number, a name, and marks.
  - These values describe the same person.

```
int roll = 42;
char name[20] = "Asha";
double marks = 72.5;
```

- With many students, how do we keep related information together?

struct

- A structure combines several named members.
- Members can have different types.

name of datatype/record

```
struct Student {
    int roll;
    char name[20];
    double marks;
};
```

- This defines what a student record contains.

Create a student record

- Declare a variable using the structure type.
- Give its members initial values.

```
struct Student st= {42, "Asha", 72.5};
```

- Initial values follow the order of the member declarations.

st :

Member	Initial value
roll	42
name	"Asha"
marks	72.5

Accessing a created record

- Use the variable name, a dot, and the member name.

```
printf("Roll: %d\n", st.roll);
printf("Name: %s\n", st.name);
printf("Marks: %.1f\n", st.marks);
```

- Each member can be used like an ordinary variable of its type.

Modifying data in a record

- A corrected score changes only the marks member.

```
student.marks = 80.0;
printf("%s: %.1f\n",st.name,st.marks);
```

- Other members remain unchanged

- Q: Can you change the name? st.name=“Divya” ?

Copying a record (assignment operator)

- Structure assignment copies all members.
- The two records can then be changed independently.

```
struct Student original = {42, "Asha", 72.5};
struct Student revised = original;

revised.marks = 80.0;

printf("%.1f\n", original.marks);
printf("%.1f\n", revised.marks);
```

